

DGrid AI 2

Security Assessment

CertiK Assessed on Sept 28th, 2026





Certik Assessed on Sept 28th, 2026

DGrid AI 2

The security assessment was prepared by Certik.

Executive Summary

TYPES

Staking

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

DGrid AI

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 09/12/2026

Final report published on 09/29/2026

Vulnerability Summary

36

Total Findings

5

Resolved

0

Partially Resolved

31

Acknowledged

0

Declined

2 Centralization

2 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

1 Major

1 Resolved



Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

8 Medium

3 Resolved, 5 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

17 Minor

1 Resolved, 16 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

8 Informational

8 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | DGRID AI 2

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[Core Contracts](#)

[ChainlinkPriceFeed](#)

[DGAI](#)

[tDGAI](#)

[Dgrid](#)

[DgridNode](#)

[DgridStakePool](#)

[DGAISTaking](#)

[DgridTopUp](#)

[DgridPriceFeed](#)

[DgridAIArena](#)

[External Dependencies](#)

[Privileged Functions](#)

Findings

[DGA-01 : Centralization Related Risks](#)

[DGA-02 : Initial Token Distribution](#)

[DGA-03 : Unpriced Multi-Asset Node Purchase](#)

[DGA-04 : Fixed Rewards Accrue Through Paused Periods](#)

[DGA-05 : TickMath Rounding Error Causes Incorrect `tDGAI` TWAP Accounting](#)

[DGA-06 : Fee-On-Transfer Tokens Inflate Inbound Accounting](#)

[DGA-07 : Incomplete Server Signature Binding](#)

[DGA-08 : Inconsistent Reward Accrual Controls During Suspension](#)

[DGA-09 : Post-Pre-Claim TDGAI Remains Accruable But Unclaimable](#)

[DGA-10 : Tier-Weight Accounting Inconsistency Due To Truncation](#)

[DGA-39 : Stale Cached Prices Can Block Legitimate Oracle Updates And Native Top-Ups](#)

[DGA-11 : Failing Reward Token Blocks Other Claims](#)

[DGA-12 : Same-Block Cache Can Return An Older Oracle Round After A Feed Update](#)

[DGA-13 : Oracle Rotation Keeps Stale Cache From The Previous Feed](#)

[DGA-14 : Deviation Check Underestimates Upward Price Moves](#)

[DGA-15 : Accepted Boundary Ticks Are Rejected After Computation](#)

[DGA-16 : Restaking Can Deposit Rewards Into A Jailed Node](#)

[DGA-17 : Insufficient Parameter Bounds Validation](#)

[DGA-18 : Unsafe ERC721 Minting Without Receiver Callback](#)

[DGA-19 : Potential Low TWAP Tick Truncates `TDGAI` Price To Zero](#)

[DGA-20 : Mutable `TDGAI` Corrupts Reward Accounting](#)

[DGA-21 : Unsynchronized `NODE\\_ID` May Break Migrated Purchases](#)

[DGA-22 : Accepted Sale Assets Cannot Be Removed](#)

[DGA-23 : Missing `handlePreStakeCall` After Pre-Stake](#)

[DGA-24 : Unvalidated `setFeeRecipient` Parameters](#)

[DGA-25 : Unbounded Pending-Unstake Queues May Lock Principal And Rewards](#)

[DGA-26 : Reward Restaking Bypasses Global And Position Claim Cooldowns](#)

[DGA-40 : Sub-18-Decimal TDGAI Payments Are Under-Credited](#)

[DGA-27 : Redundant Code](#)

[DGA-28 : Cooling-Period Commission Depends On The Unstaker To Settle](#)

[DGA-29 : Missing Emit Events](#)

[DGA-30 : Unpause Event Misnames The Resulting Pause-State Value](#)

[DGA-31 : `updatePool` Discards Accumulator Division Remainders](#)

[DGA-33 : Jailed Nodes Continue Accruing Delegator Rewards And Owner Commissions](#)

[DGA-36 : Pre-Claim UID Binding And Allocation Logic](#)

[DGA-38 : Node Owners Earn Delegation Commission On Their Own Stake](#)

Optimizations

[DGA-32 : Missing `IDGAIStaking` Interface Prevents Compilation](#)

Appendix

Disclaimer

CODEBASE | DGRID AI 2

Repository

<https://github.com/dgridai/DGRID>

Commit

- [9b730c6285569465630cb40a6e2c1b51e50ca7c6](#)
- [f19a47de23864ee717440594e0a16f4cb2a98522](#)

Audit Scope

The files in scope are listed in the appendix.

APPROACH & METHODS | DGRID AI 2

This audit was conducted for DGrid AI to evaluate the security and correctness of the smart contracts associated with the DGrid AI 2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of DGrid AI.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | DGRID AI 2

Overview

The **DGrid AI** protocol combines node issuance, NFT-based node staking, token staking, reward distribution, account top-ups, arena rewards, and oracle-based asset pricing through a set of independently deployed modules.

Core Contracts

ChainlinkPriceFeed

The contract provides normalized asset prices using configured Chainlink-compatible aggregators. Each supported asset is associated with an aggregator and its reported decimal precision. `fetchPrice()` retrieves the latest round, validates that the answer and timestamp are present, applies a configurable heartbeat, scales the result to 18 decimals, and caches it for the current block. A deviation threshold compares each newly retrieved value with the previously cached price before the cache is updated. Consumers such as `Dgrid` and `DgridTopUp` use the resulting values when converting native currency or other supported assets into protocol accounting units. The owner can replace individual aggregator addresses and update the heartbeat and maximum price-deviation parameters. Unlike the protocol's upgradeable modules, this contract uses constructor-based deployment and standard ownership.

DGAI

The contract is the primary ERC-20 token used by the DGAI staking system. Its constructor establishes the token name, symbol, maximum supply, initial supply, and owner. The initial token allocation is minted directly to the designated owner at deployment. Subsequent issuance is performed through an owner-only `mint()` function, with cumulative supply constrained by the immutable deployment-time supply policy represented by `maxSupply`. The token otherwise follows the standard ERC-20 transfer, approval, and allowance model. Within the wider architecture, DGAI is deposited into `DGAISTaking` as principal, distributed as staking rewards, charged as a fee by `DgridStakePool`, and may be transferred between the node staking and token staking modules when users elect to restake rewards.

tDGAI

The contract implements a capped ERC-20 token with constructor-based ownership. Deployment parameters define its metadata, supply cap, initial supply, and administrative owner. The initial supply is assigned to the owner, while additional units may be minted through the owner-only `mint()` function until the configured cap is reached. The token serves as a protocol asset in the broader DGrid AI architecture, including balance accounting in `DgridStakePool` and payment processing in `DgridTopUp`. When configured as a top-up asset, its USD-denominated value is obtained from the PancakeSwap V3 TWAP-based `DgridPriceFeed`, rather than being treated as a fixed-value stablecoin. The token uses conventional ERC-20 balances and allowances and does not implement custom transfer hooks within this module.

Dgrid

The contract coordinates node sales and the issuance of node NFTs. A purchase request identifies the order, beneficiary, optional parent account, node quantity, expiration time, and payment asset. The order parameters must be authorized by a signature from the configured server, and each order identifier can be fulfilled once. The payment value is derived from the

configured USD-denominated node price, together with a per-node gas amount, and converted into the selected ERC-20 asset using its decimal precision and an oracle price. After collection, the referral portion is retained as claimable commission for the parent, while the remaining amount is transferred to the development address. The contract then assigns sequential identifiers and instructs `DgridNode` to mint the purchased NFTs. Owner controls manage pricing, supported assets, oracle and module addresses, commission settings, pausing, and withdrawals.

DgridNode

The contract represents protocol nodes as upgradeable ERC-721 tokens. Minting is restricted to the configured `Dgrid` purchasing contract, which supplies the recipient and sequential node identifier after a completed purchase. The configured `DgridStakePool` can update separate staked and jailed state flags for batches of token IDs. These states are integrated into the token update path so that transferability can be controlled according to node participation status. The owner may also globally enable or disable public transfers, independently of the staking pool's node-state management. Ownership queries and status mappings allow the staking module to verify node ownership and operational state without taking custody through a conventional NFT transfer. Administrative configuration includes the authorized purchasing contract, staking pool, and public-transfer setting.

DgridStakePool

The contract manages NFT node staking and distributes one or more ERC-20 reward tokens using block-based accumulated-reward-per-share accounting. Users deposit owned `DgridNode` token IDs, increasing their stake according to the number of nodes represented. Rewards are accrued across configurable reward tokens and can be harvested directly or routed into `DGAISTaking` as a new DGAI position. The module also tracks tDGAi-related inbound and outbound balances, pre-claim staking states, unpaid rewards, and optional fixed-reward accounting. A designated server authorizes signed deposits, node jailing and unjailing, pre-claim operations, and airdrop processing. Node status changes are propagated to `DgridNode`. The owner manages reward assets and emission rates, the server and staking-module addresses, fee configuration, withdrawal availability, pause state, and emergency token recovery.

DGAISTaking

The contract provides DGAI delegation, staking-node accounting, configurable lock tiers, and time-based reward distribution. Protocol staking nodes are created by the owner and receive user deposits across independently tracked duration tiers. Each tier has a configurable rate used to derive its accounting weight, while rewards accrue through per-second accumulated-reward-per-share calculations. Users may stake directly, enter through a server-authorized treasury-funded pre-stake flow, transfer positions between staking nodes, or convert accrued rewards into additional positions. Unstaking creates delayed withdrawal requests, and claims are tracked separately for delegator rewards, cooling-period rewards, and node-owner commissions. Nodes may be jailed or unjailed through server-controlled operations. The owner configures reward emissions, tier rates, commission and cooling parameters, pool availability, operational addresses, pause status, and emergency withdrawals.

DgridTopUp

The contract records account top-ups in a normalized 18-decimal USD accounting unit. A caller specifies the credited user, payment asset, and amount. Native-currency payments are valued through `ChainlinkPriceFeed`; tDGAi payments use the TWAP value returned by `DgridPriceFeed`; and other supported ERC-20 assets are treated as stable-value tokens and normalized according to their decimal precision. Received native currency or tokens are forwarded directly to the configured

development address, while the calculated USD amount is accumulated in `userTopUpAmount` for the designated user. The owner controls the supported-token registry, recipient address, tDGAi address, both oracle references, and pause state. When paused, the owner can invoke the emergency withdrawal mechanism for assets held by the contract.

DgridPriceFeed

The contract calculates an 18-decimal tDGAi price from a configured PancakeSwap V3 pool. It reads cumulative ticks over a configurable observation period, derives the arithmetic mean tick with negative-value rounding toward negative infinity, and converts the resulting tick into a token exchange ratio using the local `TickMath` and `FullMath` libraries. The calculation accounts for the decimal precision of both pool assets and conditionally inverts the ratio based on whether tDGAi is token zero or token one. Initialization verifies that the selected pool includes the configured tDGAi token and sets a default 30-minute TWAP period. The owner may replace the pool, update the tDGAi address, or change the observation period. `DgridTopUp` consumes this module when valuing tDGAi-denominated payments.

DgridAIArena

The contract distributes prefunded ERC-20 rewards for externally determined arena rounds. Users first register their participation through `activate()`. A configured server subsequently submits a round identifier and a list of winners, reward tokens, and amounts. Each round may be uploaded once, and allocations are added to the winners' claimable balances while aggregate token obligations are tracked in `totalOwed`. Activated users claim all available rewards across the registered reward-token list in one transaction. The contract also exposes the difference between its recorded obligations and current token balance for each reward asset. The owner can add reward tokens, update the authorized server, pause or resume operations, and withdraw held reward-token balances through the paused-state emergency withdrawal function.

External Dependencies

The **DGrid AI** protocol relies on a few external dependencies to fulfill the needs of its business logic.

The following are third-party dependency contracts used within the contract:

- @openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol
- @openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol
- @openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol
- @openzeppelin/contracts-upgradeable/token/ERC721/ERC721Upgradeable.sol
- @openzeppelin/contracts/token/ERC20/ERC20.sol
- @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol
- @openzeppelin/contracts/token/ERC721/ERC721.sol
- @openzeppelin/contracts/utils/cryptography/ECDSA.sol
- @openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol
- @openzeppelin/contracts/utils/introspection/IERC165.sol
- @openzeppelin/contracts/access/Ownable.sol
- @openzeppelin/contracts/proxy/transparent/ProxyAdmin.sol
- @openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol
- Chainlink-compatible `AggregatorV3Interface` price-feed contracts

- PancakeSwap V3 pool contracts implementing `IPancakeV3Pool`

It is assumed that these functions are trusted and implemented properly within the whole project.

Additionally, it is important to note that the audit scope excludes tokens not created by this platform; these are treated as third-party dependencies. Similarly, external contracts such as the Chainlink aggregators, PancakeSwap V3 liquidity pool, configured ERC-20 reward and payment tokens, treasury, server-operated signing infrastructure, and proxy administration components, which were created or operated by third parties, are also outside the scope of this review.

I Privileged Functions

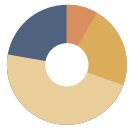
In the **DGrid AI** protocol, some privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the **Centralization Related Risks** and **Initial Token Distribution** findings.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the `TimeLock` contract.

FINDINGS | DGRID AI 2



36

Total Findings

0

Critical

2

Centralization

1

Major

8

Medium

17

Minor

8

Informational

This report has been prepared for DGrid AI to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 36 issues were identified. Leveraging a combination of DGrid AI the following findings were uncovered:

ID	Title	Category	Severity	Status
DGA-01	Centralization Related Risks	Centralization	Centralization	● Acknowledged
DGA-02	Initial Token Distribution	Centralization	Centralization	● Acknowledged
DGA-03	Unpriced Multi-Asset Node Purchase	Logical Issue	Major	● Resolved
DGA-04	Fixed Rewards Accrue Through Paused Periods	Logical Issue	Medium	● Acknowledged
DGA-05	TickMath Rounding Error Causes Incorrect <code>tDGAI</code> TWAP Accounting	Design Issue, Logical Issue	Medium	● Resolved
DGA-06	Fee-On-Transfer Tokens Inflate Inbound Accounting	Logical Issue	Medium	● Acknowledged
DGA-07	Incomplete Server Signature Binding	Logical Issue	Medium	● Resolved
DGA-08	Inconsistent Reward Accrual Controls During Suspension	Logical Issue, Inconsistency	Medium	● Acknowledged
DGA-09	Post-Pre-Claim TDGAI Remains Accruable But Unclaimable	Logical Issue	Medium	● Acknowledged
DGA-10	Tier-Weight Accounting Inconsistency Due To Truncation	Logical Issue	Medium	● Acknowledged

ID	Title	Category	Severity	Status
DGA-39	Stale Cached Prices Can Block Legitimate Oracle Updates And Native Top-Ups	Logical Issue	Medium	● Resolved
DGA-11	Failing Reward Token Blocks Other Claims	Logical Issue	Minor	● Acknowledged
DGA-12	Same-Block Cache Can Return An Older Oracle Round After A Feed Update	Logical Issue	Minor	● Acknowledged
DGA-13	Oracle Rotation Keeps Stale Cache From The Previous Feed	Logical Issue	Minor	● Acknowledged
DGA-14	Deviation Check Underestimates Upward Price Moves	Design Issue	Minor	● Acknowledged
DGA-15	Accepted Boundary Ticks Are Rejected After Computation	Logical Issue	Minor	● Acknowledged
DGA-16	Restaking Can Deposit Rewards Into A Jailed Node	Logical Issue	Minor	● Acknowledged
DGA-17	Insufficient Parameter Bounds Validation	Volatile Code	Minor	● Acknowledged
DGA-18	Unsafe ERC721 Minting Without Receiver Callback	Volatile Code	Minor	● Acknowledged
DGA-19	Potential Low TWAP Tick Truncates <code>tDGAI</code> Price To Zero	Logical Issue	Minor	● Acknowledged
DGA-20	Mutable <code>tDGAI</code> Corrupts Reward Accounting	Volatile Code	Minor	● Acknowledged
DGA-21	Unsynchronized <code>NODE_ID</code> May Break Migrated Purchases	Volatile Code	Minor	● Acknowledged
DGA-22	Accepted Sale Assets Cannot Be Removed	Logical Issue	Minor	● Acknowledged
DGA-23	Missing <code>handlePreStakeCall</code> After Pre-Stake	Logical Issue	Minor	● Acknowledged

ID	Title	Category	Severity	Status
DGA-24	Unvalidated <code>setFeeRecipient</code> Parameters	Volatile Code	Minor	● Acknowledged
DGA-25	Unbounded Pending-Unstake Queues May Lock Principal And Rewards	Logical Issue	Minor	● Acknowledged
DGA-26	Reward Restaking Bypasses Global And Position Claim Cooldowns	Logical Issue	Minor	● Acknowledged
DGA-40	Sub-18-Decimal TDGAI Payments Are Under-Credited	Logical Issue	Minor	● Resolved
DGA-27	Redundant Code	Coding Style	Informational	● Acknowledged
DGA-28	Cooling-Period Commission Depends On The Unstaker To Settle	Design Issue	Informational	● Acknowledged
DGA-29	Missing Emit Events	Coding Style	Informational	● Acknowledged
DGA-30	Unpause Event Misnames The Resulting Pause-State Value	Volatile Code, Inconsistency	Informational	● Acknowledged
DGA-31	<code>updatePool</code> Discards Accumulator Division Remainders	Logical Issue, Inconsistency	Informational	● Acknowledged
DGA-33	Jailed Nodes Continue Accruing Delegator Rewards And Owner Commissions	Design Issue	Informational	● Acknowledged
DGA-36	Pre-Claim UID Binding And Allocation Logic	Design Issue	Informational	● Acknowledged
DGA-38	Node Owners Earn Delegation Commission On Their Own Stake	Logical Issue	Informational	● Acknowledged

DGA-01 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization		● Acknowledged

Description

In the contract `ChainlinkPriceFeed`, the role `owner` has authority over the following functions:

- `setPriceFeed()`
- `setHeartbeat()`
- `setMaxPriceDeviation()`

Any compromise to the `owner` account may allow an attacker to replace trusted oracle feeds or manipulate the heartbeat and deviation thresholds governing price validation.

In the contract `DGAI`, the role `owner` has authority over the following functions:

- `mint()`

Any compromise to the `owner` account may allow an attacker to mint tokens up to the configured supply cap and distribute them to arbitrary accounts.

In the contract `DGAISTaking`, the role `owner` has authority over the following functions:

- `createStakingNodeByOwner()`
- `setCoolingTeamClaimDay()`
- `setCommissionRate()`
- `setCoolClaimDay()`
- `setPoolPerSecondReward()`
- `addFixedRate()`
- `setPoolEnabled()`
- `setServer()`
- `setDev()`
- `emergencyWithdraw()`
- `pause()`
- `unpause()`

Any compromise to the `owner` account may allow an attacker to create staking nodes, manipulate reward and commission parameters, enable or disable pools, replace operational accounts, withdraw surplus reward funds, or halt and resume staking operations.

In the contract `DGAStaking`, the role `server` has authority over the following functions:

- `handlePreStake()`
- `jailNode()`
- `unjailNode()`

Any compromise to the `server` account may allow an attacker to authorize fraudulent treasury-funded pre-stakes or arbitrarily jail and unjail staking nodes, affecting their availability and reward participation.

In the contract `Dgrid`, the role `server` has authority over the following functions:

- `buyNode()`

Any compromise to the `server` account may allow an attacker to authorize fraudulent node purchases, control node recipients and referral data, and cause node NFTs to be minted under unauthorized orders.

In the contract `Dgrid`, the role `owner` has authority over the following functions:

- `setCommissionRate()`
- `setServer()`
- `setDev()`
- `setPriceFeed()`
- `setAssets()`
- `setNodePrice()`
- `setGasAmountPerNode()`
- `pause()`
- `unpause()`
- `emergencyWithdraw()`

Any compromise to the `owner` account may allow an attacker to manipulate node pricing, commissions, payment assets, oracle and operational addresses, halt node sales, or withdraw contract-held assets while the protocol is paused.

In the contract `DgridAIArena`, the role `server` has authority over the following functions:

- `uploadWinners()`

Any compromise to the `server` account may allow an attacker to assign arbitrary rewards to activated accounts and consume round identifiers with fraudulent winner data.

In the contract `DgridAIArena`, the role `owner` has authority over the following functions:

- `addRewardToken()`
- `pause()`
- `unpause()`
- `emergencyWithdraw()`

- `setServer()`

Any compromise to the `owner` account may allow an attacker to alter the supported reward-token set, halt claims, replace the authorized server, or withdraw all reward-token balances while the arena is paused.

In the contract `DgridNode`, the role `owner` has authority over the following functions:

- `setDgridStakePool()`
- `setDgrid()`
- `setPublicTransferEnabled()`

Any compromise to the `owner` account may allow an attacker to replace trusted protocol components with malicious contracts and enable or disable public NFT transfers. A malicious replacement could consequently access protocol-only minting, staking, or jail-management operations.

In the contract `DgridPriceFeed`, the role `owner` has authority over the following functions:

- `setTDGAIPool()`
- `setTDGAI()`
- `setTWAPPeriod()`

Any compromise to the `owner` account may allow an attacker to replace the token or liquidity pool used for pricing and manipulate the TWAP observation period, potentially corrupting prices consumed by the protocol.

In the contract `DgridStakePool`, the role `owner` has authority over the following functions:

- `addRewardToken()`
- `setStakingAddress()`
- `setRewardPerBlock()`
- `setStartBlock()`
- `setServer()`
- `setRewardPerBlockByIndex()`
- `airdrop()`
- `setRewardTokenEnabled()`
- `pause()`
- `unpause()`
- `setFeeRecipient()`
- `emergencyWithdraw()`
- `emergencyWithdrawByToken()`
- `setUnstakeEnabled()`
- `setTdgaiToken()`

Any compromise to the `owner` account may allow an attacker to alter reward assets and emission rates, manipulate the staking start and availability settings, replace trusted protocol and server addresses, distribute arbitrary airdrops, redirect fees, halt the pool, or withdraw contract-held tokens while paused.

In the contract `DgridStakePool`, the role `server` has authority over the following functions:

- `deposit()`
- `jailNodes()`
- `unjailNodes()`

Any compromise to the `server` account may allow an attacker to authorize staking or unjailing operations through forged approvals and directly jail users' nodes, thereby manipulating staking eligibility and reward accrual.

In the contract `DgridTopUp`, the role `owner` has authority over the following functions:

- `addSupportedToken()`
- `setDev()`
- `setTDGAI()`
- `setTDGridPriceFeed()`
- `setChainLinkPriceFeed()`
- `pause()`
- `unpause()`
- `emergencyWithdraw()`
- `deleteSupportedToken()`

Any compromise to the `owner` account may allow an attacker to manipulate supported payment tokens, replace treasury and pricing dependencies, halt top-up operations, remove supported assets, or withdraw contract-held funds while paused.

In the contract `tDGAi`, the role `owner` has authority over the following functions:

- `mint()`

Any compromise to the `owner` account may allow an attacker to mint tokens up to the configured supply cap and distribute them to arbitrary accounts.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multi-signature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised; AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement. AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles. OR
- Remove the risky functionality.

I Alleviation

[DGrid AI, 09/14/2026]:

The owner role is required for protocol administration and emergency operations, including configuration updates, pausing and unpausing the contract, and emergency handling. The owner authority is controlled by a Safe multisignature wallet, and privileged operations require approval from multiple authorized signers.

The ownership of the DGAI contract has been permanently renounced and transferred to the zero address.

We acknowledge that compromise of the server signing key could authorize unauthorized operations. This role is required by the current protocol design, and the associated centralization risk is accepted as a known trust assumption.

[Certik, 09/15/2026]:

It is suggested to implement the aforementioned methods to mitigate centralized failure. Also, Certik strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

[DGrid AI, 09/17/2026]:

About server attack risk:

`DGAISTaking.handlePreStake()` is part of a time-limited migration campaign and relies on server-side authorization, so we acknowledge the centralized trust assumption.

The campaign is expected to end in approximately two months. After that, the remaining treasury assets will be moved to a multisig wallet, and this flow will no longer be used.

Any abnormal pre-stake activity can be monitored on-chain, and because the positions have at least a 180-day lock period, the team has sufficient time to investigate and take operational or upgrade actions before funds become withdrawable.

DGA-02 | Initial Token Distribution

Category	Severity	Location	Status
Centralization	● Centralization	contracts/Dgrid.sol (v2-9b730c6): 20; contracts/tD GAI.sol (v2-9b730c6): 20	● Acknowledged

Description

All of the tokens are sent to the contract deployer or one or several externally-owned account (EOA) addresses. This is a centralization risk because the deployer or the owner(s) of the EOAs can distribute tokens without obtaining the consensus of the community. Any compromise to these addresses may allow a hacker to steal and sell tokens on the market, resulting in severe damage to the project.

Recommendation

It is recommended that the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team should make efforts to restrict access to the private keys of the deployer account or EOAs. A multi-signature (2/3, 3/5) wallet can be used to prevent a single point of failure due to a private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize the project team with a third-party KYC provider to create greater accountability.

Alleviation

[DGrid AI, 09/14/2026]:

The allocation plan is as follows:

- Initial Liquidity: 7%, released via TGE
- Airdrop: 8%, released via TGE
- Investors: 10%, locked for 1 year, then released linearly over 2 years
- Team Incentives: 10%, locked for 1 year, then released linearly over 2 years
- Community: 15%, locked for 6 months, then released linearly over 2 years
- Node: 50%, released linearly over 10 years, halving every 2 years.

Relevant contracts:

- DGrid-Airdrop: 0xDc4E528BBbAbb0982C9eB004CCD7724AD3414B69
- DGrid-Community: 0xB8064d59F96dca9e5f6da293Bc9A04235Bb729D0
- DGrid-Initial-Liquidity: 0xc02f4CCFAece017d3d78C27b5690F70f73Fe208e
- DGrid-Investor: 0x332f68F8d6aF382570bEEb802636c3FD99b6B0E6
- DGrid-Node: 0xF85cf6D6D1e71977861c394f45CE1800Cc4A1dd9

- DGrid-Team: 0x3FC2aa472c855e2BED0E37F0450caB1581494f1a

DGA-03 | Unpriced Multi-Asset Node Purchase

Category	Severity	Location	Status
Logical Issue	● Major	contracts/Dgrid.sol (v2-9b730c6): 177~182, 261~266, 268~273	● Resolved

Description

The payment `asset` is chosen entirely by the caller. `buyNode()` only checks that the asset is on the owner-controlled allowlist; it is not included in the server-signed digest.

```
function buyNode(
    ...
    address asset
) public nonReentrant whenNotPaused {
    ...
    require(assetInfos[asset].token != address(0), "Invalid asset");
    ...
    bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(
        abi.encode(
            block.chainid,
            orderId,
            user,
            parent,
            nodeCount,
            expireTime
        )
    );
    ...
    uint256 paymentAmountInAsset = (paymentAmount *
        (10 ** assetInfos[asset].decimals)) / 1e18;
    uint256 gasAmountInAsset = (gasAmount *
        (10 ** assetInfos[asset].decimals)) / 1e18;
    uint256 totalAmountInAsset = paymentAmountInAsset + gasAmountInAsset;
    ...
}
```

Both charges are derived only from `nodeCount` and the current admin parameters and then scaled to the selected token's decimals.

```
function calculatePaymentAmount(
    uint256 nodeCount
) public view returns (uint256) {
    require(nodePrice > 0, "Node price not set");
    return nodePrice * nodeCount * 1e18;
}

function calculateGasAmount(
    uint256 nodeCount
) public view returns (uint256) {
    require(gasAmountPerNode > 0, "Gas amount not set");
    return gasAmountPerNode * nodeCount * 1e18;
}
```

For each allowlisted token, this applies only a decimal conversion. Although the contract stores a `priceFeed`, `buyNode()` never calls `fetchPrice()`, so different assets are not normalized to a common value. As a result, a user can choose a cheaper allowlisted asset and purchase the same number of nodes for a lower economic cost.

Proof of Concept

The following PoC uses Foundry to simulate a user purchasing nodes with a cheaper allowlisted token while paying the same token amount.

It allows two 18-decimal tokens with Chainlink prices of 1 (*USDT*) and 0.01 (JUNK), then uses valid server signatures to buy one node with each asset. `buyNode` charges `110e18` units of both tokens (`nodePrice + gasAmountPerNode`) and never calls `fetchPrice`. If the stored price feed were used, JUNK would cost `11000e18` tokens; the cheaper asset therefore purchases the same node at 1/100 of the oracle value.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {MessageHashUtils} from
"@openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol";

import {Dgrid} from "../contracts/Dgrid.sol";
import {DgridNode} from "../contracts/DgridNode.sol";
import {ChainlinkPriceFeed} from "../contracts/ChainlinkPriceFeed.sol";

/// @dev Proves buyNode() charges allowlisted assets by decimals only and never
calls fetchPrice.
contract BuyNodeUnpricedAssetPoC is Test {
    uint256 internal constant SERVER_PK = 1;
    address internal server;
    address internal owner = address(0xA11CE);
    address internal dev = address(0xDE5);
    address internal buyer = address(0xB0B);

    MockERC20 internal usdt; // $1
    MockERC20 internal junk; // $0.01
    Dgrid internal dgrid;
    DgridNode internal node;
    ChainlinkPriceFeed internal feed;

    function setUp() public {
        server = vm.addr(SERVER_PK);
        usdt = new MockERC20("USDT", "USDT", 18);
        junk = new MockERC20("JUNK", "JUNK", 18);

        address[] memory tokens = new address[](2);
        address[] memory aggs = new address[](2);
        tokens[0] = address(usdt);
        tokens[1] = address(junk);
        aggs[0] = address(new MockAgg(1e8)); // $1, 8 decimals
        aggs[1] = address(new MockAgg(1e6)); // $0.01, 8 decimals
        feed = new ChainlinkPriceFeed(tokens, aggs, 1 days);

        node = DgridNode(
            address(new ERC1967Proxy(address(new DgridNode()), ""))
        );
        dgrid = Dgrid(address(new ERC1967Proxy(address(new Dgrid()), "")));
        node.initialize(owner, address(dgrid), address(1));

        address[] memory assets = new address[](2);
        assets[0] = address(usdt);
```

```
assets[1] = address(junk);
dgrid.initialize(
    owner,
    server,
    dev,
    address(feed),
    address(node),
    0,
    assets,
    100, // nodePrice
    10 // gasAmountPerNode
);

usdt.mint(buyer, 1_000_000e18);
junk.mint(buyer, 1_000_000e18);
vm.startPrank(buyer);
usdt.approve(address(dgrid), type(uint256).max);
junk.approve(address(dgrid), type(uint256).max);
vm.stopPrank();
}

function test_cheaperAllowlistedAssetPaysTheSameTokenUnits() public {
    uint256 usdtPrice = feed.fetchPrice(address(usdt)); // 1e18
    uint256 junkPrice = feed.fetchPrice(address(junk)); // 1e16
    assertEq(usdtPrice, 1e18);
    assertEq(junkPrice, 1e16);

    uint256 quoted = dgrid.calculatePaymentAmount(1) +
        dgrid.calculateGasAmount(1); // 110e18

    vm.startPrank(buyer);
    dgrid.buyNode(1, buyer, address(0), 1, block.timestamp + 1, _sign(1),
address(usdt));
    dgrid.buyNode(2, buyer, address(0), 1, block.timestamp + 1, _sign(2),
address(junk));
    vm.stopPrank();

    // Both buys mint one node, but charge the same 110e18 token units.
    assertEq(node.balanceOf(buyer), 2);
    assertEq(usdt.balanceOf(dev), 110e18);
    assertEq(junk.balanceOf(dev), 110e18);
    assertEq(usdt.balanceOf(dev), junk.balanceOf(dev));

    // If fetchPrice were used, junk would cost 100x more tokens:
    // (quoted * 10**decimals) / price = 110e18 * 1e18 / 1e16 = 11000e18
    uint256 oracleJunkAmount = (quoted * (10 ** junk.decimals())) / junkPrice;
    assertEq(oracleJunkAmount, 11_000e18);
    assertLt(junk.balanceOf(dev), oracleJunkAmount);
}
```

```
function _sign(uint256 orderId) internal view returns (bytes memory) {
    bytes32 digest = MessageHashUtils.toEthSignedMessageHash(
        abi.encode(
            block.chainid,
            orderId,
            buyer,
            address(0),
            uint256(1),
            block.timestamp + 1
        )
    );
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(SERVER_PK, digest);
    return abi.encodePacked(r, s, v);
}

contract MockERC20 is ERC20 {
    uint8 private immutable _dec;

    constructor(
        string memory n,
        string memory s,
        uint8 d
    ) ERC20(n, s) {
        _dec = d;
    }

    function decimals() public view override returns (uint8) {
        return _dec;
    }

    function mint(address to, uint256 a) external {
        _mint(to, a);
    }
}

contract MockAgg {
    int256 public price;

    constructor(int256 p) {
        price = p;
    }

    function decimals() external pure returns (uint8) {
        return 8;
    }

    function latestRoundData()
```

```

    external
    view
    returns (uint80, int256, uint256, uint256, uint80)
    {
        return (1, price, 0, block.timestamp, 1);
    }
}

```

The output is:

```

Ran 1 test for test/BuyNodeUnpricedAsset.t.sol:BuyNodeUnpricedAssetPoC
[PASS] test_cheaperAllowlistedAssetPaysTheSameTokenUnits() (gas: 334394)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 9.25ms (4.35ms CPU
time)

```

Recommendation

Consider converting the 18-decimal payment and gas amounts through `priceFeed.fetchPrice(asset)` and charging that converted quantity:

```

uint256 price = priceFeed.fetchPrice(asset); // 18 decimals
require(price > 0, "invalid price");
uint256 totalAmountInAsset =
    ((paymentAmount + gasAmount) * (10 ** assetInfos[asset].decimals)) / price;

```

Alleviation

[DGrid AI, 09/17/2026]:

`buyNode()` has been changed to USDT-only. The caller can no longer select the payment asset, and the configured USDT address is included in the server-signed digest together with `chainId` and `address(this)`. Therefore, a valid signature cannot be reused with a cheaper allowlisted asset.

The payment amount is now calculated directly in USDT decimals, and oracle normalization is no longer required because multi-asset payment support was removed from this flow.

The changes have been reflected in the commit hash: [c3f9be4effcfeb760a15c36b5bbcf2a9dfe04909](#).

[CertiK, 09/17/2026]:

It should be noted that `nodePrice` and `gasAmountPerNode` are intended to be whole-number values without token decimals (e.g. `100` = 100 USDT). Please configure them accordingly so that the later `1e18` scaling and USDT-decimal conversion do not over- or under-charge.

DGA-04 | Fixed Rewards Accrue Through Paused Periods

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/DgridStakePool.sol (v2-9b730c6): 815~818, 828	● Acknowledged

Description

Fixed-token rewards continue to accrue across paused blocks, even though the normal reward-token path correctly freezes rewards during pauses.

Normal reward accounting goes through `updatePool()`. When `paused` is true, the function advances `lastRewardBlock` to the current block without increasing `accPerShares`, so no rewards accrue for the paused period.

```
if (paused) {
    // freeze rewards
    lastRewardBlock = block.number;
    return;
}
```

The fixed-reward path does not use `lastRewardBlock` or any pause-adjusted checkpoint. Instead, it relies on `fixedLatestRewardBlock[_user]`, a per-user checkpoint that is updated to `block.number` only after that user's fixed rewards are accrued or paid.

```
uint256 latestRewardBlock = fixedLatestRewardBlock[_user];
if (latestRewardBlock == 0) {
    latestRewardBlock = startBlock;
}

for (uint256 i = 0; i < fixedRewardTokenInfos.length; i++) {
    uint256 rewardBlocks = _calculateFixedRewardBlocks(
        latestRewardBlock,
        fixedRewardTokenInfos[i].finalRewardBlock
    );
    ...
}
```

`rewardBlocks` is calculated from `fixedLatestRewardBlock[_user]` to `block.number`, capped only by `finalRewardBlock`. Paused blocks are not excluded.

```
function _calculateFixedRewardBlocks(  
    uint256 _userLatestRewardBlock,  
    uint256 _finalBlock  
) internal view returns (uint256) {  
    if (block.number < startBlock) {  
        return 0;  
    }  
  
    if (_finalBlock == 0 || _finalBlock > block.number) {  
        return block.number - _userLatestRewardBlock;  
    } else {  
        if (_finalBlock <= _userLatestRewardBlock) {  
            return 0;  
        } else {  
            return _finalBlock - _userLatestRewardBlock;  
        }  
    }  
}
```

The impact is overpayment of fixed rewards for paused blocks. This can deplete fixed-reward balances faster than intended and may cause later claims to fail if the inflated rewards exceed the contract's token balance. Principal is not affected, but reward accounting and payouts for the remaining fixed-reward program can become incorrect.

Recommendation

We recommend making all fixed reward calculations pause-aware, including `pendingFixedReward()` and `_accrueFixedUnpaid()`, so paused blocks cannot be counted or paid.

Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. This finding applies to the legacy fixed-reward path. Fixed rewards have already been terminated, and the contract has switched to DGA linear emission. Future rewards are calculated through `updatePool()`, which already freezes `accPerShare` during pause by advancing `lastRewardBlock` without accruing rewards. Therefore, paused blocks cannot inflate current DGA rewards, and we will not change the deprecated fixed-reward accounting.

DGA-05 | TickMath Rounding Error Causes Incorrect tDGAi TWAP Accounting

Category	Severity	Location	Status
Design Issue, Logical Issue	● Medium	contracts/libraries/TickMath8.sol (v2-9b730c6): 64~66	● Resolved

Description

`TickMath8.getSqrtRatioAtTick()` uses an incorrect rounding addend when converting the internal Q128.128 ratio to Q64.96. Instead of rounding the 32-bit shift by at most one Q96 unit, it adds `2^128 - 1` before shifting right by 32 bits. At `tick == 0`, this causes the function to return approximately `2^97` rather than the canonical `2^96`.

```
// Q128 -> Q96
sqrtPriceX96 = uint160(
    (ratio + 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF) >> 32
);
```

This library output is used by `DgridPriceFeed.getTDGAITwapPrice18()`, which reads the UniswapV3 TWAP tick from `pool.observe()` and passes it into `getPriceFromTick()`. Because `getPriceFromTick()` squares the inflated sqrt price, the returned price is deterministically incorrect.

The issue affects `DgridTopUp.topUp()` when users pay with `tDGAi`. The function transfers the caller's `tDGAi` to `dev` and records `userTopUpAmount[user]` using the incorrect TWAP price, which may cause recorded USD top-up amounts to be either over-credited or under-credited.

Recommendation

Consider replacing the conversion with the canonical Uniswap V3 rounding formula and restore compatible `TickMath` boundary constants.

Alleviation

[DGrid AI, 09/15/2026]:

`DgridTopUp` currently cannot process `tDGAi` top-ups because `DgridPriceFeed` is not set on the contract, so the priceFeed address is still zero. We will deploy `DgridPriceFeed` and set it on `DgridTopUp`. In addition, `TickMath` previously used an incorrect `Q128.128` to `Q64.96` rounding formula, which inflated `sqrtPriceX96` and caused `getPriceFromTick()` to return an incorrect `tDGAi` TWAP. We have replaced it with the canonical Uniswap V3 rounding formula and restored the original `TickMath` boundary constants.

Changes have been reflected in the commit: [bfd3c1ee95e577c01f3194ac3968588e731d3e3e](#).

[CertiK, 09/24/2026]:

The `Q128.128` to `Q64.96` conversion now uses the canonical round-up, so `getSqrtRatioAtTick(0)` returns `2^96`. The boundary constants are still shifted inward (`MIN_SQRT_RATIO = 4295128739 + 1` , `MAX_SQRT_RATIO` one below the Uniswap value), so `getSqrtRatioAtTick()` still reverts at both extreme ticks.

[DGrid AI, 09/24/2026]:

We agree that the `TickMath` boundary constants should stay consistent with the Uniswap V3 standard. We have updated `MIN_SQRT_RATIO` and `MAX_SQRT_RATIO` to the official Uniswap V3 values to avoid unnecessary reverts at the extreme ticks.

The changes have been reflected in the commit: [f19a47de23864ee717440594e0a16f4cb2a98522](#).

DGA-06 | Fee-On-Transfer Tokens Inflate Inbound Accounting

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/Dgrid.sol (v2-9b730c6): 193; contracts/DgridTopUp.sol (v2-9b730c6): 140	● Acknowledged

Description

`Dgrid.buyNode()` and `DgridTopUp.topUp()` pull in caller-specified ERC20 amounts, then credit ledger balances from that same `amount`. Neither path measures the actual received balance. Owner-whitelisted assets are not checked for transfer fees.

Dgrid.buyNode():

```
commissionAmount = (paymentAmountInAsset * commissionRate) / 100;
commission[parent][asset] += commissionAmount;
_safeTransferFrom(asset, msg.sender, address(this), totalAmountInAsset);
_safeTransfer(asset, dev, totalAmountInAsset - commissionAmount);
```

DgridTopUp.topUp():

```
usdAmount = (amount * price) / 1e18; // or _adjust18Decimals(amount, decimals)
_safeTransferFrom(token, msg.sender, dev, amount);
userTopUpAmount[user] += usdAmount;
```

Fee-on-transfer tokens may return success from `transferFrom()` while the recipient receives less than the specified `amount`. If the contract still records the full `amount`, its internal ledger can become higher than the actual token balance.

As these operations accumulate, the recorded withdrawable or claimable amount may exceed the contract's real holdings. Later withdrawals, reward claims, or settlements may then revert due to insufficient balance, preventing the contract from honoring the recorded amounts.

Recommendation

Consider rejecting fee-on-transfer tokens when adding payment assets, or credit only the recipient's balance increase after the transfer.

Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. The supported assets are standard ERC20 tokens such as USDT, which do not charge transfer fees. Fee-on-transfer tokens are not used and will not be added to the whitelist. Therefore, this issue does not affect the current

production assets.

DGA-07 | Incomplete Server Signature Binding

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/Dgrid.sol (v2-9b730c6): 157~166; contracts/DgridStakePool.sol (v2-9b730c6): 349~358, 456~465	● Resolved

Description

`Dgrid.buyNode()`

The server signature currently binds only `chainId`, `orderId`, `user`, `parent`, `nodeCount`, and `expireTime`. It does not bind several values that materially affect how the order is executed, including `asset`, `address(this)`, `nodePrice`, and `gasAmountPerNode`.

```
bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(
    abi.encode(
        block.chainid,
        orderId,
        user,
        parent,
        nodeCount,
        expireTime
    )
);
```

Because these fields are not part of the signed digest, they remain determined by the caller or by mutable contract state at execution time:

- `asset`: The payment token is passed in by the caller and is only checked against the owner allowlist. It should be fixed by the signed order so the user cannot choose a different allowlisted token at execution time.
- `nodePrice` and `gasAmountPerNode`: These values can be changed by the owner. The executed payment should use the prices that were intended when the server issued the signature, rather than whatever values are live when the transaction is submitted.
- `address(this)`: The signed order is not bound to a specific contract instance. If the same `server` key is reused across multiple `Dgrid` deployments on the same chain, an unused order may be accepted by another instance because `fulfilledOrders` is tracked per contract.

`DgridStakePool.deposit()` / `DgridStakePool.unjailNodes()`

The server signatures for `deposit()` and `unjailNodes()` do not bind `address(this)`, so the signed authorization is not tied to a specific `DgridStakePool` instance.

```

bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(
    abi.encode(
        block.chainid,
        _nodes,
        _staker,
        _expireTime,
        ACTION_DEPOSIT,
        _nonce
    )
);

bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(
    abi.encode(
        block.chainid,
        _nodeIds,
        _owner,
        _expireTime,
        ACTION_UNJAIL,
        _nonce
    )
);

```

Because `address(this)` is omitted from the signed digest, a valid signature issued for one `DgridStakePool` deployment may also be accepted by another deployment on the same chain if the same `server` key is reused. The nonce mappings are stored per contract, so a nonce consumed in one deployment does not prevent the same signed message from being used in another.

Recommendation

Consider improving the signed message contents to bind the intended contract instance and other caller-controlled parameters.

Alleviation

[DGrid AI, 09/17/2026]:

The `buyNode()` signature has been strengthened. The signed digest now includes `chainId`, `address(this)`, the configured USDT asset, `orderId`, `user`, `parent`, `nodeCount`, and `expireTime`.

The server-signed digests for `deposit()` and `unjailNodes()` now include `address(this)`. This binds each authorization to the specific `DgridStakePool` contract instance and prevents the same signature from being replayed across different deployments on the same chain. The backend signing logic has been updated accordingly to sign the target `DgridStakePool` proxy address.

The changes have been reflected in the commit: [c3f9be4effc7b760a15c36b5bbcf2a9dfe04909](#).

[Certik, 09/17/2026]:

In the latest update, `buyNode()` still does not bind `nodePrice` or `gasAmountPerNode` in the signed digest. The executed payment therefore uses the live admin-configured values at transaction time, not the prices that were in effect when the server issued the signature. If those parameters are changed after signing but before the order is submitted, the user may be charged a different amount than the signed order implied.

We would like to confirm whether this is intended by design.

[DGrid AI, 09/21/2026]: Acknowledged, we have updated the `buyNode()` signature digest to include both `nodePrice` and `gasAmountPerNode` in the commit: 96835d5b64c41a8a6fdc87570b7ef4c4f7d3ef03.

DGA-08 | Inconsistent Reward Accrual Controls During Suspension

Category	Severity	Location	Status
Logical Issue, Inconsistency	● Medium	contracts/DGAStaking.sol (v2-9b730c6): 1203~1206, 1301	● Acknowledged

Description

The `DGAStaking` contract applies inconsistent suspension controls to active-stake rewards and pending-unstake rewards.

Active-stake rewards are accrued through `_updatePool`, which stops node-pool reward accumulation while the protocol is paused or `poolInfos[NODE_POOL_ID].enabled` is set to `false`.

```
function _updatePool(uint8 _poolId) internal {
    PoolInfo storage pool = poolInfos[_poolId];
    uint256 nowTs = block.timestamp;
    if (nowTs <= pool.lastRewardTime) {
        return;
    }

    if (paused || !pool.enabled || pool.totalStaked == 0) {
        pool.lastRewardTime = uint64(nowTs);
        return;
    }
    ...
}
```

In contrast, `_pendingUnstakeUnpaid` calculates the reward for a pending unstake based on elapsed wall-clock time until the selected tier unlocks, without accounting for either suspension condition.

```

function _pendingUnstakeUnpaid(
    PendingUnstake storage _req
) internal view returns (uint256 rewardUnpaid, uint256 commissionUnpaid) {
    uint256 reward = 0;
    if (_req.amount > 0) {
        uint256 endTime = _req.unStakeTime + (uint256(_req.day) * 1 days);
        uint256 rewardTime = block.timestamp < endTime
            ? block.timestamp
            : endTime;
        if (rewardTime > _req.unStakeTime) {
            reward =
                (_req.amount *
                    uint256(fixedRates[_req.day]) *
                    (rewardTime - _req.unStakeTime)) /
                (BPS_DENOMINATOR * 365 days);
        }
    }
    rewardUnpaid = reward > _req.claimedReward
        ? reward - _req.claimedReward
        : 0;
    ...
}

```

As a result, once a user submits an unstake request, the request continues accumulating its fixed cooling-period reward even while active node-pool reward accrual is suspended. The user can later claim this reward after the request unlocks. Although the reward is capped according to the selected tier, the contract still creates additional node-reward obligations during periods in which the corresponding active-stake reward mechanism has been stopped.

This inconsistent accounting means that pausing the protocol or disabling the node pool does not fully suspend node-related reward generation. Users who have moved their stake into the pending-unstake state continue earning rewards, while users whose stake remains active do not.

Recommendation

Apply consistent accrual controls to active-stake and pending-unstake rewards. Pending-unstake rewards should not accrue while the protocol is paused or the node pool is disabled, consistent with active-stake rewards.

Alleviation

[DGrid AI, 09/17/2026]:

Under normal operation, the `DGAStaking` contract is expected to remain unpaused, and pool disabling or protocol pause is reserved for exceptional administrative or emergency situations.

We acknowledge that pending-unstake rewards currently continue to accrue based on wall-clock time, while active-stake rewards are suspended during pause or pool disable. If we need to pause the protocol or disable the node pool in the future,

we will upgrade `DGAStaking` before or together with that operation to make pending-unstake rewards follow the same suspension rules as active-stake rewards.

For the current production state, we acknowledge and accept the residual risk.

DGA-09 | Post-Pre-Claim TDGAI Remains Accruable But Unclaimable

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/DgridStakePool.sol (v2-9b730c6): 641	● Acknowledged

Description

`preClaimAndStake()` records the user's full `getTdgaiAvailable()` amount in `tdgaiTransferOut()` and sets `preStaked = 1`, but does not stop further `tdGAI` accrual or remove the user's stake share from the pool.

```
function preClaimAndStake(
    uint8 _index,
    uint8 _cid,
    string memory _uid
) external {
    require(paused, "!paused");
    require(_index < 3, "index is out of range");
    require(_cid == 0 || _cid == 1, "cid is out of range");
    require(preStaked[msg.sender] == 0, "pre staked already");
    ...
    preStaked[msg.sender] = 1;
    tdgaiTransferOut[msg.sender] += amount;
    tdgaiTransferIn[address(0)] += amount;
    emit PreClaimAndStake(
        msg.sender,
        _uid,
        _cid,
        _index,
        preClaimAmount,
        stakeAmount
    );
}
```

After unpause, `updatePool()` can keep accruing `tdGAI` into unpaid rewards, and `airdrop` can increase `tdgaiTransferIn`. `getTdgaiAvailable()` may therefore show a positive balance again. There is no user path to withdraw that balance as ERC-20 (`harvest` skips `tdgaiToken`), and `preClaimAndStake()` cannot be repeated. Post-conversion `tdGAI` credits can remain permanently stuck in accounting.

Recommendation

Consider stopping further `tdGAI` credits for that user after pre-claim (accrual cutoff and reject/settle post-claim airdrops), or providing an explicit settlement path for newly accrued `tdGAI`.

■ Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. tDGAi rewards will not be re-enabled, and `preClaimAndStake()` will be deprecated. This function was only used for the one-time paused migration. After unpause, rewards are distributed in DGAi, and `harvest()` does not pay tDGAi. We will not change this deprecated path.

DGA-10 | Tier-Weight Accounting Inconsistency Due To Truncation

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/DGAIStaking.sol (v2-9b730c6): 1277~1285	● Acknowledged

Description

`_tierWeight()` calculates a position's weight using integer division:

```
function _tierWeight(uint64 _day, uint256 _amount) internal view returns (uint256) {
    if (_amount == 0) {
        return 0;
    }
    return (_amount * uint256(fixedRates[_day])) / RATE_BASE;
}
```

Because the result is rounded down, calculating the weight of each individual transaction is not equivalent to calculating the weight of the aggregated position. In other words, the sum of `_tierWeight()` across multiple tranches may differ from the weight calculated over their combined amount.

However, `_applyStake()` and `unstake()` update `nodeWeightCache` and `poolInfos[NODE_POOL_ID].totalStaked` using the individually rounded weight of each operation:

`_applyStake()`:

```
uint256 weight = _tierWeight(_day, _amount);
pos.amount += _amount;
nodeWeightCache[_nodeId] += weight;
poolInfos[NODE_POOL_ID].totalStaked += weight;
```

`unstake()`:

```
uint256 weight = _tierWeight(_day, _amount);
pos.amount -= _amount;
pos.pendingUnstake += _amount;
nodeWeightCache[_nodeId] -= weight;
poolInfos[NODE_POOL_ID].totalStaked -= weight;
```

Meanwhile, position-level reward accounting calculates weight from the aggregated `pos.amount`. This inconsistency causes `nodeWeightCache` and the pool's `totalStaked` to diverge from the actual aggregate position weights when users split deposits or make partial withdrawals.

A similar inconsistency exists in `changeNodeByUser()`. When an amount is merged into an existing target position, the function credits `_tierWeight(day, moveAmount)` instead of calculating the difference between the target position's weight

before and after the merge.

The resulting accounting drift can cause node commission rewards to be calculated from an incorrect cached weight. It can also cause withdrawals or node migrations to subtract more weight than was previously recorded, potentially reverting the operation and blocking the affected user's exit. If other users' cached weight prevents an immediate underflow, the incorrect subtraction may instead reduce the shared `totalStaked` below the aggregate weight of the remaining positions, disrupting subsequent pool reward accrual and withdrawals.

Recommendation

Update cached node and pool weights using the difference between each position's weight before and after every principal change, including staking, unstaking, reward restaking, and node migration. Existing cached values should also be reconciled, with tests covering rounding boundaries, split deposits, partial withdrawals, full exits, and position merges.

Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. DGA1 has 18 decimals, so any `_tierWeight()` rounding difference is at most 1 wei per operation and is economically immaterial. We will not change the live weight-cache accounting in this release and accept the residual rounding inconsistency.

DGA-39 | Stale Cached Prices Can Block Legitimate Oracle Updates And Native Top-Ups

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/ChainlinkPriceFeed.sol (v2-9b730c6): 70	● Resolved

Description

The adapter can repeatedly reject a fresh, valid oracle price because it compares it with an arbitrarily old cached price. This can suspend native-token top-ups even though Chainlink is reporting current data.

`fetchPrice` checks the new observation's `updatedAt` against `heartbeat`, but uses `lastPrice` as the deviation baseline without checking `lastTimestamp`. If the deviation exceeds `MAX_PRICE_DEVIATION`, it reverts before updating the cache, so every retry repeats the same stale comparison. With the default 50% threshold, rejection occurs when the new price is more than twice or less than half the cached price; exactly double or half is allowed because the comparison is strict.

After sufficient inactivity and a qualifying market move, public refreshes for the affected asset and native-token calls to `DgridTopUp.topUp` revert. ERC-20 top-ups remain available, and the condition can be resolved if the market returns within range or a trusted owner adjusts the deviation threshold or replaces the adapter. The denial is therefore not permanent, but it can continue indefinitely pending market movement or administrative intervention.

Recommendation

Apply the deviation check only when `lastTimestamp[asset]` is recent enough to be a meaningful baseline, such as within the configured heartbeat; otherwise, accept the fresh observation as a new baseline. This preserves the invariant that deviation checks compare temporally related prices and that valid fresh oracle data can replace an expired cache.

Alleviation

[DGrid AI, 09/17/2026]:

The deviation check now only applies when the cached price is still recent enough to be a meaningful baseline.

Specifically, `fetchPrice()` verifies that `lastTimestamp[asset]` is within the configured heartbeat before comparing the fresh Chainlink price against `lastPrice[asset]`. If the cached price is stale, the fresh Chainlink observation is accepted as the new baseline after passing the normal heartbeat and validity checks. This prevents valid fresh oracle data from being repeatedly rejected due to an arbitrarily old cached price.

The changes have been reflected in the commit hash: [c3f9be4effcfb760a15c36b5bbcf2a9dfe04909](#).

DGA-11 | Failing Reward Token Blocks Other Claims

Category	Severity	Location	Status
Logical Issue	● Minor	contracts/DgridAIArena.sol (v2-9b730c6): 124; contracts/DgridStakePool.sol (v2-9b730c6): 490, 572	● Acknowledged

Description

`DgridAIArena` and `DgridStakePool` both pay rewards in a single batch and do not provide a way to claim rewards per token. `DgridAIArena.claimReward()` iterates through all configured reward tokens. `DgridStakePool.harvest()` similarly pays all non-`tDGA` unpaid rewards through `_payReward()`, then calls `_payFixedReward()` in the same transaction.

If any token transfer in the batch reverts, the entire claim reverts. Previous accounting updates and transfers in the same transaction are rolled back, so the recorded rewards are not lost, but no rewards are paid. Since the caller cannot skip the failing token, otherwise claimable and funded rewards remain blocked until that token can be transferred successfully again.

This can happen if one reward token is underfunded, temporarily non-transferable, blacklists the recipient, or otherwise reverts during transfer. The issue affects reward-claim liveness but does not directly affect principal.

Recommendation

Consider adding a per-token claim function to avoid a single failing reward token blocking the caller from claiming otherwise funded rewards.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-12 | Same-Block Cache Can Return An Older Oracle Round After A Feed Update

Category	Severity	Location	Status
Logical Issue	Minor	contracts/ChainlinkPriceFeed.sol (v2-9b730c6): 77	Acknowledged

Description

`ChainlinkPriceFeed` can return an older cached Chainlink price for the rest of the current block after the underlying aggregator has already advanced to a newer round.

The root cause is the early return in `fetchPrice()`: when `lastBlock[asset]` equals `block.number` and `lastPrice[asset]` is nonzero; the function returns the cached value without checking `latestRoundData` or the latest `roundId`. Any caller can populate this cache before a Chainlink update transaction in the same block. Later calls in that block, including `DgridTopUp.topUp()` for native BNB payments, then reuse the pre-update price even though the aggregator now reports a newer price.

In `DgridTopUp.topUp`, the returned BNB price is used to compute `usdAmount` and permanently increment `userTopUpAmount[user]`. If the cached price is higher than the updated price, the user can be over-credited for the BNB sent; if it is lower, the user is under-credited.

Recommendation

We recommend removing the same-block cache or validating that the cached round is not stale by comparing the stored `lastRoundId` against the aggregator's current round. If a cache is kept, at minimum include a staleness check that reverts if the cached block is older than the current block or if the aggregator has moved to a newer round within the same block.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-13 | Oracle Rotation Keeps Stale Cache From The Previous Feed

Category	Severity	Location	Status
Logical Issue	Minor	contracts/ChainlinkPriceFeed.sol (v2-9b730c6): 142	Acknowledged

Description

Replacing a price feed in `ChainlinkPriceFeed.setPriceFeed()` updates `priceFeeds` and `feedDecimals` but does not clear the per-asset price cache.

```
function setPriceFeed(address asset, address priceFeed) external onlyOwner {
    priceFeeds[asset] = AggregatorV3Interface(priceFeed);
    feedDecimals[asset] = priceFeeds[asset].decimals();
}
```

If the old feed was already cached in the current block, `fetchPrice()` returns `lastPrice` without reading the newly configured feed.

```
if (lastBlock[asset] == block.number && lastPrice[asset] != 0) {
    return lastPrice[asset];
}
```

In later blocks, the first price from the new feed is compared against `lastPrice` from the old feed. A difference above `MAX_PRICE_DEVIATION` (default 50%) reverts before the cache can be updated.

```
if (lastPrice[asset] != 0 && lastRoundId[asset] != 0) {
    uint256 minP = scaledPrice < lastPrice[asset] ? scaledPrice : lastPrice[asset];
    uint256 maxP = scaledPrice > lastPrice[asset] ? scaledPrice : lastPrice[asset];
    uint256 deviation = ((maxP - minP) * 1e18) / maxP;
    if (deviation > MAX_PRICE_DEVIATION) revert PriceFeed__Deviation();
}
```

This can cause one-block stale pricing after an oracle rotation and can also make the asset's oracle path revert until governance changes configuration or the new price falls within the old-feed deviation window. `DgridTopUp.topUp` uses `fetchPrice(address(0))` for native-token payments, so stale or unavailable oracle data can affect user top-up accounting.

Recommendation

We recommend deleting the cached price data for the asset when replacing a price feed. If the new feed should be validated immediately, warm the cache from the new feed after the old values are cleared.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-14 | Deviation Check Underestimates Upward Price Moves

Category	Severity	Location	Status
Design Issue	● Minor	contracts/ChainlinkPriceFeed.sol (v2-9b730c6): 100	● Acknowledged

Description

`ChainlinkPriceFeed.fetchPrice` calculates price deviation using the higher of the cached price and the newly reported price as the denominator. As a result, the configured 50% deviation limit does not actually cap upward price changes at 50% relative to the cached price. For example, a cached price of 100 can be updated to 200 because the contract evaluates the deviation as $(200 - 100) / 200 = 50\%$.

```
if (lastPrice[asset] != 0 && lastRoundId[asset] != 0) {
    uint256 minP = scaledPrice < lastPrice[asset]
        ? scaledPrice
        : lastPrice[asset];
    uint256 maxP = scaledPrice > lastPrice[asset]
        ? scaledPrice
        : lastPrice[asset];
    @> uint256 deviation = ((maxP - minP) * 1e18) / maxP;
    if (deviation > MAX_PRICE_DEVIATION) revert PriceFeed__Deviation();
}
```

For native BNB payments, `topUp` calls `chainLinkPriceFeed.fetchPrice(address(0))` and then credits `userTopUpAmount[user]` as $\text{msg.value} * \text{bnbPrice} / 1e18$. Any user can trigger this flow while the contract is not paused, provided the Chainlink round is fresh, positive, and not future-dated. If an upward price movement passes the understated deviation check, `fetchPrice` updates `lastPrice`, and the same top-up transaction records credit using the newly accepted price.

The practical concern is that an unexpected or manipulated upward oracle move of up to almost 100% from the cached price may still pass a guard configured as 50%. This can inflate BNB-denominated top-up credit beyond what the previous-price limit appears intended to allow.

Recommendation

Consider computing $\text{abs}(\text{newPrice} - \text{oldPrice}) * 1e18 / \text{oldPrice}$ and apply the threshold to that value.

Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. `ChainlinkPriceFeed` is currently only used as a placeholder/auxiliary oracle module and is not actively relied upon by the live production flow. This issue does not affect current accounting, payments, or rewards. We will not

change it in this release and accept the residual risk. If the oracle becomes actively used in the future, we will revisit the cache invalidation logic.

DGA-15 | Accepted Boundary Ticks Are Rejected After Computation

Category	Severity	Location	Status
Logical Issue	Minor	contracts/libraries/TickMath8.sol (v2-9b730c6): 17, 67~71	Acknowledged

Description

The `TickMath` helper accepts the standard minimum and maximum ticks in its initial tick bounds check, but can still revert after computing the sqrt ratio for those same boundary inputs.

The root cause is that `TickMath8` shifts both sqrt-ratio constants inward: `MIN_SQRT_RATIO` is one greater than the standard value and `MAX_SQRT_RATIO` is one less than the standard value. Because `getSqrtRatioAtTick` then performs a second post-computation range check, `getSqrtRatioAtTick(-887272)` and `getSqrtRatioAtTick(887272)` compute the standard boundary ratios and fail the final `"S"` check.

This path is reachable through `DgridPriceFeed.getTDGAIWapPrice18()`, which derives a TWAP tick from `pool.observe()` and passes it to `getPriceFromTick()`. `DgridTopUp.topUp()` calls this price feed before transferring `tDGAi` or updating `userTopUpAmount`, so a boundary TWAP tick would revert `tDGAi` top-ups without leaving partial state changes.

Recommendation

We recommend making the accepted `tick` range and `sqrt-ratio` range consistent at both boundaries. Either use the standard `MIN_SQRT_RATIO` and `MAX_SQRT_RATIO` returned by `getSqrtRatioAtTick(MIN_TICK)` and `getSqrtRatioAtTick(MAX_TICK)`, or narrow the accepted `tick` range so boundary ticks that would fail the post-computation check are rejected earlier.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-16 | Restaking Can Deposit Rewards Into A Jailed Node

Category	Severity	Location	Status
Logical Issue	● Minor	contracts/DGAStaking.sol (v2-9b730c6): 612	● Acknowledged

Description

A jailed node remains eligible as a `restakedDGA` destination. The function verifies only that the target ID exists, then calls `_applyStake` without checking that `stakingNodeMap[_targetNodeId].nodeStatus` is active.

This bypasses the node-jail admission control used by normal staking, prestaking, external reward restaking, and node migration. A user can direct rewards into a node that the server has jailed, where the new principal continues participating in reward and commission accounting. A jailed node owner can therefore continue growing stake and receiving associated commission through this alternate deposit path.

Recommendation

Before calling `_applyStake`, load the target node and require `targetNode.nodeStatus == 1`. This preserves the invariant enforced by all other deposit paths.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-17 | Insufficient Parameter Bounds Validation

Category	Severity	Location	Status
Volatile Code	Minor	contracts/ChainlinkPriceFeed.sol (v2-9b730c6): 63, 63, 147, 148, 151, 154; contracts/DGAIStaking.sol (v2-9b730c6): 775, 780, 785; contracts/Dgrid.sol (v2-9b730c6): 123; contracts/DgridStakePool.sol (v2-9b730c6): 174, 503, 539	Acknowledged

Description

Several owner-controlled parameters lack consistent lower/upper bound checks between `initialize` / `constructor` and their corresponding setters. Misconfiguration during deployment or later updates can write invalid values, break expected business logic, miscalculate rewards or commissions, or cause critical paths to revert.

Dgrid — `commissionRate`

`initialize()` assigns `commissionRate` without an upper bound, while `setCommissionRate()` enforces `<= 100`. If `commissionRate` is initialized above `100` and a `parent` is present, `buyNode()` reverts on underflow when transferring the dev share.

```
function initialize(
    ...
    commissionRate = _commissionRate;
    ...
}
```

DgridStakePool — `startBlock`

`initialize()` allows `startBlock` to be set in the past, while `setStartBlock()` requires `>= block.number`. If `startBlock` is set too early at deployment, the pool may accrue a large amount of historical block rewards at once.

```
function initialize(
    ...
    startBlock = _startBlock;
    ...
}
```

DgridStakePool — `rewardPerBlock`

`initialize()` and `addRewardToken()` require `rewardPerBlock > 0`, but `setRewardPerBlock()` and `setRewardPerBlockByIndex()` do not. The owner can set per-block rewards to zero and stop emissions for a reward token.

```
function setRewardPerBlock(
    uint256[] memory _rewardPerBlock
) external onlyOwner {
    ...
    rewardTokenInfos[i].rewardPerBlock = _rewardPerBlock[i];
    ...
}

function setRewardPerBlockByIndex(
    uint256 _rewardTokenIndex,
    uint256 _rewardPerBlock
) external onlyOwner {
    ...
    rewardTokenInfos[_rewardTokenIndex].rewardPerBlock = _rewardPerBlock;
}
```

DGAStaking — minimum stake amounts

`initialize()` requires all three minimum stake amounts to be greater than zero, but the three setters accept zero. Later staking or delegation may bypass the intended minimum thresholds.

```
function setMinLlmStakeAmount(uint256 _minAmount) external onlyOwner {
    emit SetMinLlmStakeAmount(minLlmStakeAmount, _minAmount);
    minLlmStakeAmount = _minAmount;
}

function setMinDelegatorStakeAmount(uint256 _minAmount) external onlyOwner {
    emit SetMinDelegatorStakeAmount(minDelegatorStakeAmount, _minAmount);
    minDelegatorStakeAmount = _minAmount;
}

function setMinNodeSelfStakeAmount(uint256 _minAmount) external onlyOwner {
    emit SetMinNodeSelfStakeAmount(minNodeSelfStakeAmount, _minAmount);
    minNodeSelfStakeAmount = _minAmount;
}
```

ChainlinkPriceFeed — heartbeat / MAX_PRICE_DEVIATION

The constructor, `setHeartbeat`, and `setMaxPriceDeviation` enforce no lower bounds. Setting either value to zero causes `fetchPrice` to revert on nearly all non-zero price updates.

```
constructor(  
    address[] memory _tokens,  
    address[] memory _aggregators,  
    uint32 _heartbeat  
) Ownable(msg.sender) {  
    ...  
    heartbeat = _heartbeat;  
    ...  
}  
  
function setHeartbeat(uint32 _heartbeat) external onlyOwner {  
    heartbeat = _heartbeat;  
}  
  
function setMaxPriceDeviation(  
    uint256 _maxPriceDeviation  
) external onlyOwner {  
    MAX_PRICE_DEVIATION = _maxPriceDeviation;  
}
```

Recommendation

Apply consistent and reasonable lower/upper bound checks to the same parameter in both initialization and setter paths to prevent misconfiguration.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-18 | Unsafe ERC721 Minting Without Receiver Callback

Category	Severity	Location	Status
Volatile Code	Minor	contracts/DgridNode.sol (v2-9b730c6): 62	Acknowledged

Description

`DgridNode.mint()` mints node NFTs with `_mint()` instead of `_safeMint()`. `Dgrid.buyNode()` calls this function and sends the token to `user`, which comes from a server-signed order and is not restricted to an EOA.

```
function mint(address to, uint256 id) external onlyDgrid {
    require(to != address(0), "Invalid to");
    _mint(to, id);
    emit Mint(to, id);
}
```

The `_mint()` does not call `onERC721Received()`. If `user` is a contract that cannot handle ERC721 tokens, the mint still succeeds, and the node NFT can be locked in that contract. OpenZeppelin discourages `_mint()` for this reason and recommends `_safeMint()` whenever the recipient may be a contract.

Recommendation

Consider replacing `_mint(to, id)` with `_safeMint(to, id)` in `DgridNode.mint()`, so minting reverts when the recipient is a contract that does not implement `IERC721Receiver`.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-19 | Potential Low TWAP Tick Truncates tDGAi Price To Zero

Category	Severity	Location	Status
Logical Issue	Minor	contracts/DgridTopUp.sol (v2-9b730c6): 129, 140	Acknowledged

Description

`DgridPriceFeed.getPriceFromTick()` computes the 18-decimal price with two floor `FullMath.mulDiv()` calls. At a sufficiently low tick, the intermediate result becomes 0, including ticks that still pass `TickMath.getSqrtRatioAtTick()`.

```
uint256 base = FullMath.mulDiv(1e18 * (10 ** decimals0), sqrtP, Q96);
return FullMath.mulDiv(base, sqrtP, Q96 * (10 ** decimals1));
```

`getTDGAITwapPrice18()` then uses that value as follows:

- If `token1 == tDGAi`, it executes `(1e36) / price`. A zero price reverts the call, so the `tDGAi` topUp path reverts.
- If `token0 == tDGAi`, it returns 0. `DgridTopUp.topUp` does not require a positive tDGAi price (unlike the BNB branch). It still transfers tDGAi to `dev` and adds 0 to `userTopUpAmount`.

This is integer truncation in the pricing formula, which triggers when the TWAP tick is extremely low.

Additionally, a second zero-credit path exists for supported stablecoins with `decimals > 18`: `_adjust18Decimals` floors via division, so any `amount < 10 ** (decimals - 18)` yields `usdAmount = 0`. Payment is still transferred to `dev` with no `usdAmount > 0` check.

Recommendation

Consider adding the requirement `usdAmount > 0` after every USD conversion and before transfer to ensure payment always records a nonzero top-up credit.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. TDGAi is deprecated. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-20 | Mutable tDGAI Corrupts Reward Accounting

Category	Severity	Location	Status
Volatile Code	Minor	contracts/DgridStakePool.sol (v2-9b730c6): 496, 575~577, 722~725	Acknowledged

Description

DgridStakePool supports multiple reward tokens. On each `harvest()`, `_payReward()` iterates through the configured reward tokens and transfers the user's accrued balance for every token except the address currently stored in `tdgaiToken`. Rewards denominated in `tdgaiToken` are therefore kept as internal balances instead of being paid out as ERC20 tokens. The owner can update `tdgaiToken` at any time through `setTdgaiToken()`.

```
function _payReward(address _user) internal {
    UserInfo storage user = userInfo[_user];
    for (uint256 i = 0; i < rewardTokenInfos.length; i++) {
        if (address(rewardTokenInfos[i].rewardToken) == tdgaiToken) {
            continue; //skip tdgai reward token
        }
        ...
    }
}
```

The issue is that `_payReward()` decides whether a reward should be withheld using the current `tdgaiToken` value at harvest time, while accrued rewards do not record which token had this role when they were earned. As a result, changing `tdgaiToken` can retroactively change how outstanding rewards are settled. For example, if users accrue token A while A is configured as `tdgaiToken`, those balances remain internal. If the owner later changes `tdgaiToken` to token B while token A is still configured as a reward token, `_payReward()` will no longer exclude token A, allowing users to harvest previously withheld token A balances as normal ERC20 rewards.

As a result, an authorized token-role update can unintentionally release historical rewards that were originally meant to remain under internal-balance accounting.

Recommendation

We recommend binding the internal-balance treatment to accrued rewards at the time they are recorded, or restricting `setTdgaiToken()` so it cannot be used once rewards have started accruing unless a dedicated migration procedure is performed first.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-21 | Unsynchronized `NODE_ID` May Break Migrated Purchases

Category	Severity	Location	Status
Volatile Code	Minor	contracts/Dgrid.sol (v2-9b730c6): 92, 200~202; contracts/DgridNode.sol (v2-9b730c6): 135	Acknowledged

Description

The `Dgrid` contract sells nodes and mints them as NFTs. During `initialize()`, it accepts an existing `DgridNode` collection address and calls `_initNodeNodeId()`, which unconditionally resets the mint counter `NODE_ID` to `1`.

`buyNode()` then mints each purchased node through `dgridNode.mint(user, NODE_ID)` and increments the counter. On the NFT side, `setDgrid()` allows the collection owner to authorize a new `Dgrid` contract as the minter, which appears to be the intended path for replacing the purchase contract while keeping the same NFT collection.

However, nothing synchronizes `NODE_ID` with the existing collection. If a new `Dgrid` contract is initialized against a non-empty `DgridNode` collection, its first `buyNode()` call will attempt to mint token ID `1`. If that token ID already exists, the ERC721 mint reverts. The transaction rolls back and no funds are lost, but `NODE_ID` never advances, so every later purchase fails in the same way. Since `NODE_ID` has no setter, the purchase flow remains unavailable until the contract is upgraded or replaced with a correctly synchronized counter.

Recommendation

Synchronize the mint counter during migration using an authoritative next-token ID maintained by `DgridNode`, or initialize `Dgrid` with a starting ID that is proven to be greater than every token ID already minted in the retained collection.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-22 | Accepted Sale Assets Cannot Be Removed

Category	Severity	Location	Status
Logical Issue	Minor	contracts/Dgrid.sol (v2-9b730c6): 319	Acknowledged

Description

`Dgrid.setAssets()` can add new payment assets, but it does not provide any way to revoke or remove a previously accepted asset from `assetInfos` and `assetList`.

```
function setAssets(address[] memory _assets) public onlyOwner {
    for (uint256 i = 0; i < _assets.length; i++) {
        require(_assets[i] != address(0), "Invalid asset");
        assetInfos[_assets[i]] = Asset({
            token: _assets[i],
            decimals: ERC20(_assets[i]).decimals()
        });

        // remove duplicate assets
        bool exists = false;
        for (uint256 j = 0; j < assetList.length; j++) {
            if (assetList[j] == _assets[i]) {
                exists = true;
                break;
            }
        }
        if (!exists) assetList.push(_assets[i]);
    }
    emit SetAssets(_assets);
}
```

As a result, if an accepted token later becomes deprecated, misconfigured, or otherwise unsuitable for node purchases, the owner cannot cleanly disable it through the sale contract's own asset-management flow.

Recommendation

Consider adding a removal function for sold assets or otherwise providing a clear mechanism to disable previously accepted assets.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-23 | Missing `handlePreStakeCall` After Pre-Stake

Category	Severity	Location	Status
Logical Issue	Minor	contracts/DGAStaking.sol (v2-9b730c6): 383; contracts/DgridStakePool.sol (v2-9b730c6): 681	Acknowledged

Description

`DgridStakePool.preClaimAndStake()` sets `preStaked[user] = 1`. Only `handlePreStakeCall()` may advance that flag to `2`, and it is restricted to the `DGAStaking` contract.

```
function handlePreStakeCall(address _user) external {
    require(msg.sender == dgaiStaking, "not DgaiStaking address");
    require(checkPreStaked(_user) == 1, "invalid pre staked already");
    preStaked[_user] = 2;
    emit HandlePreStake(_user);
}
```

However, `DGAStaking.handlePreStake()` completes the treasury-funded stake, but never calls `handlePreStakeCall()`. The pool therefore remains at state `1` after a successful stake.

Recommendation

Consider calling `DgridStakePool.handlePreStakeCall(user)` in `DGAStaking.handlePreStake()`.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-24 | Unvalidated `setFeeRecipient` Parameters

Category	Severity	Location	Status
Volatile Code	Minor	contracts/DgridStakePool.sol (v2-9b730c6)	Acknowledged

Description

`setFeeRecipient()` lacks validation for `_feeRecipient` and `_feeAmount`.

```
function setFeeRecipient(
    address _feeRecipient,
    uint256 _feeAmount
) external onlyOwner {
    feeRecipient = _feeRecipient;
    feeAmount = _feeAmount;
}
```

When `_feeRecipient` is set to `address(0)`, the fee transfer logic is silently disabled; when `_feeAmount` is set too large with no upper bound, `_payReward()` requires each `DGAI` payout to strictly exceed that amount, which may cause small or even all `DGAI` claims to permanently revert.

Recommendation

Consider adding a non-zero address validation for `_feeRecipient` and setting a reasonable upper bound for `_feeAmount`.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-25 | Unbounded Pending-Unstake Queues May Lock Principal And Rewards

Category	Severity	Location	Status
Logical Issue	Minor	contracts/DGAIStaking.sol (v2-9b730c6): 750, 1336	Acknowledged

Description

Each partial `unstake` creates a separate pending request and appends its ID to `userPendingUnstakeIds`, indexed by node, user, and lock tier. The contract does not enforce a maximum number of outstanding requests or aggregate compatible requests.

Reward settlement depends on processing these arrays in full. `_accruePendingUnstakeReward` traverses every pending request in a tier, while `_accrueUnpaid` invokes this settlement across all tiers held by the user under a node. Consequently, operations such as further unstaking and claiming unstaked principal must settle the existing queues before proceeding. `claimUnstake` additionally performs a linear search to remove the selected request, and `changeNodeByUser` processes and deletes the pending requests for the migrated tiers.

Although creating another request already requires traversing the existing queue, successful creation does not guarantee that subsequent settlement will remain executable. During queue construction, pending rewards may round to zero, allowing the contract to skip corresponding storage updates. After rewards accumulate, processing the same queue can require additional storage writes for many requests and exceed the block gas limit.

A transaction that exhausts its gas reverts before releasing principal or removing requests, leaving the queue unchanged. The contract provides no bounded settlement or cleanup mechanism that allows the user to make incremental progress. As a result, cooling principal and rewards associated with the oversized queue can become inaccessible. Remaining active principal under the same node can also become inaccessible because further unstaking and node migration invoke the same settlement path.

Recommendation

Enforce a conservative limit on outstanding requests per user and node, aggregating compatible requests or rejecting additional partial unstakes when the limit is reached. Refactor settlement to process individual requests or bounded batches with persisted progress, and maintain request indexes for constant-time removal. Provide a bounded recovery mechanism for existing oversized queues; adding a cap alone does not resolve queues already exceeding safe processing limits.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-26 | Reward Restaking Bypasses Global And Position Claim Cooldowns

Category	Severity	Location	Status
Logical Issue	Minor	contracts/DGAIStaking.sol (v2-9b730c6): 471, 612, 790	Acknowledged

Description

`claim` enforces both the protocol-wide settlement start time and the source position's cooldown before accruing or consuming rewards.

However, `restakeDGAi` does not enforce the same restrictions. It can accrue a user's active-position and pending-unstake rewards, clear the unpaid reward balance, mark the amount as claimed, and convert it into new active staking principal even when an ordinary claim would still be rejected.

The converted amount increases the target position's principal and future reward entitlement. Users can therefore compound rewards before the configured settlement period, participate in future reward distributions earlier than intended, and potentially dilute the rewards allocated to users who remain subject to the cooldown.

Recommendation

Apply the same global and per-position settlement checks to `restakeDGAi` before any reward accrual or consumption. The shared validation should preferably be implemented in an internal helper used by both `claim` and `restakeDGAi` to prevent the two settlement paths from diverging.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-40 | Sub-18-Decimal TDGAI Payments Are Under-Credited

Category	Severity	Location	Status
Logical Issue	Minor	contracts/DgridTopUp.sol (v2-9b730c6): 100	Resolved

Description

When a token is configured as `tDGAi`, `DgridTopUp.topUp()` obtains its USD price from `DgridPriceFeed`, which returns an 18-decimal price. However, the contract multiplies the raw token amount directly by that price:

```
usdAmount = (amount * price) / 1e18;
```

The amount is not normalized according to `supportedTokensInfos[token].decimals`, even though the token's decimals are recorded during `setTDGAI()`. Consequently, tokens with fewer than 18 decimals are under-credited by a factor of $10^{(18 - \text{tokenDecimals})}$.

For example, if a six-decimal token is configured as `tDGAi` and priced at `$1`, transferring one token uses `amount = 1e6`. The calculation records only `1e6` USD units instead of the expected `1e18` USD units. Nevertheless, the full `1e6` token units are transferred from the user to `dev` through `_safeTransferFrom()`. The discrepancy is then added to `userTopUpAmount[user]` and emitted in the `TopUp` event.

The issue is reachable because `setTDGAI()` accepts any ERC-20 token and records its runtime decimals, while `DgridPriceFeed` supports pool tokens with up to 18 decimals. Although the project's current `tDGAi` implementation uses the default 18 decimals, a valid configured replacement with fewer decimals triggers the issue for every subsequent top-up. Users can therefore pay the full token amount while receiving substantially less recorded USD credit, potentially causing a direct loss of value or an underfunded account balance in any downstream system relying on `userTopUpAmount` or the emitted top-up data.

Recommendation

Normalize the raw tDGAi amount to 18 decimals using `supportedTokensInfos[token].decimals` before multiplying it by the 18-decimal TWAP. This preserves the invariant that `userTopUpAmount` uses the same 18-decimal USD unit regardless of the payment token's native precision.

Alternatively, restrict `setTDGAI()` to tokens with exactly 18 decimals and enforce that requirement explicitly.

Alleviation

[DGrid AI, 09/24/2026]:

Issue acknowledged. The changes have been reflected in the commit hash: [340b1f0ffe9aa2e4169771dca7c9cf1174abcc80](#).

DGA-27 | Redundant Code

Category	Severity	Location	Status
Coding Style	● Informational	contracts/DGAStaking.sol (v2-9b730c6); contracts/DgridStakePool.sol (v2-9b730c6)	● Acknowledged

Description

The following functions are commented out or left as unused empty implementations:

DGAStaking

- `stakeLlm()`
- `unstakeLlm()`
- `claimTeamReward()`
- `claimLlm()`
- `claimUnstakeLlm()`
- `updateTeamPool()`
- `updateLlmPool()`
- `pendingLlmReward()`
- `pendingTeamReward()`
- `setMinLlmStakeAmount()`
- `setLlmCommissionRate()`
- `_accrueLlmUnpaid()`
- `_accrueTeamUnpaid()`
- `_resetLlmDebt()`
- `_resetTeamDebt()`

DgridStakePool

- `addFixedRewardToken()`
- `setFixedRewardTokenFinalRewardBlock()`
- `setFixedRewardPerBlock()`
- `endFixedTokenReward()`
- `transferTdgai()`
- `batchTransferTdgai()`

Recommendation

Remove these redundant functions to reduce code surface. When upgrading, keep existing state variables in place (including unused ones) so the storage layout and slot ordering remain unchanged.

I Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I won't make any changes for the current version.

DGA-28 | Cooling-Period Commission Depends On The Unstaker To Settle

Category	Severity	Location	Status
Design Issue	● Informational	contracts/DGAIStaking.sol (v2-9b730c6): 1329, 1430	● Acknowledged

Description

A node owner cannot independently realize commission earned while a staker's principal is in the cooling period.

`_accrueNodeCommission` only accounts for `nodeWeightCache`, which excludes pending unstakes; when active weight is zero, it resets the debt and returns.

Cooling-period commission is added only by `_accruePendingUnstakeReward()`, reached through later actions for the former staker. `claimNodeCommission` does not settle pending requests, and `pendingNodeCommission` omits them when active weight is zero. A staker can leave an expired pending-unstake request unclaimed indefinitely, withholding the corresponding earned commission from the node owner.

```
function _accruePendingUnstakeReward(
    uint64 _nodeId,
    address _user,
    uint64 _day
) internal {
    uint256[] storage ids = userPendingUnstakeIds[_nodeId][_user][_day];
    Position storage pos = positions[_nodeId][_user][_day];
    for (uint256 i = 0; i < ids.length; i++) {
        PendingUnstake storage req = pendingUnstake[_nodeId][_user][ids[i]];
        (
            uint256 rewardUnpaid,
            uint256 commissionUnpaid
        ) = _pendingUnstakeUnpaid(req);
        if (rewardUnpaid > 0) {
            pos.unpaid += rewardUnpaid;
            req.claimedReward += rewardUnpaid;
        }
        if (commissionUnpaid > 0) {
            nodeCommissionUnpaid[_nodeId] += commissionUnpaid;
            req.claimedCommission += commissionUnpaid;
        }
    }
}
```

Recommendation

Consider adding a permissionless per-request settlement path that credits the commission portion of a pending unstake to its capped end time, and allow the node owner to call it before claiming commission.

I Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-29 | Missing Emit Events

Category	Severity	Location	Status
Coding Style	● Informational	contracts/ChainlinkPriceFeed.sol (v2-9b730c6): 142, 147, 151; contracts/Dgrid.sol (v2-9b730c6): 296; contracts/DgridPriceFeed.sol (v2-9b730c6): 86, 94, 102; contracts/DgridStakePool.sol (v2-9b730c6): 200, 722; contracts/DgridTopUp.sol (v2-9b730c6): 254	● Acknowledged

Description

There should always be events emitted in sensitive functions that are controlled by centralization roles.

Recommendation

It is recommended to emit events in sensitive functions that are controlled by centralization roles.

Alleviation

[DGrid AI, 09/15/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-30 | Unpause Event Misnames The Resulting Pause-State Value

Category	Severity	Location	Status
Volatile Code, Inconsistency	● Informational	contracts/Dgrid.sol (v2-9b730c6): 357~360	● Acknowledged

Description

The `Unpause` event labels its Boolean parameter `unpaused`, although the emitted value represents the contract's resulting `paused` state.

After `unpause()` sets `paused` to false, it emits `Unpause(msg.sender, false)`. The value is therefore correct for the pause-state convention used throughout the project, but consumers relying on the ABI parameter name may interpret false as meaning that unpausing failed and record the opposite operational state.

Recommendation

Rename the `Unpause` event's Boolean parameter from `unpaused` to `paused`.

Alleviation

[DGrid AI, 09/16/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-31 | `updatePool` Discards Accumulator Division Remainders

Category	Severity	Location	Status
Logical Issue, Inconsistency	● Informational	contracts/DgridStakePool.sol (v2-9b730c6): 245	● Acknowledged

Description

`DgridStakePool.updatePool()` updates each reward token's accumulator using integer division, but does not carry the modulo remainder into future updates.

```
uint256 reward = blocks * rewardTokenInfos[i].rewardPerBlock;  
accPerShares[i] += (reward * ACC_PRECISION) / totalStaked;
```

Whenever `(reward * ACC_PRECISION) % totalStaked != 0`, the scaled remainder is discarded. Since `DgridStakePool` does not keep remainder-carry storage or any dust accounting, repeated `updatePool()` calls can leave part of the configured emission permanently excluded from `accPerShares`. As a result, stakers may receive slightly less than the nominal `rewardPerBlock` emission over time.

The same repository handles this pattern more precisely in `DGAISTaking._updateGroup()`: it adds the previously stored remainder to the next scaled reward calculation and stores the new modulo for future updates.

```
uint256 scaled = uint256(group.perSecondReward) *  
    dt *  
    ACC_PRECISION +  
    accRewardRemainders[_groupId];  
deltaAcc = scaled / group.totalStaked;  
accRewardRemainders[_groupId] = scaled % group.totalStaked;
```

Recommendation

Consider carrying forward division remainders in `updatePool()`, consistent with `DGAISTaking`, so configured emissions are fully reflected in `accPerShares` over time.

Alleviation

[DGrid AI, 09/16/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

DGA-33 | Jailed Nodes Continue Accruing Delegator Rewards And Owner Commissions

Category	Severity	Location	Status
Design Issue	● Informational	contracts/DGAIStaking.sol (v2-9b730c6): 902, 914	● Acknowledged

Description

Jailing only changes `nodeStatus`; it does not suspend the node's reward accounting. Existing positions on a jailed node continue to accrue through the global node accumulator, and the node owner continues to accrue commission.

```
// node jail and unjail , user can choose a node to stake
function jailNode(uint64 _nodeId) external whenServer nodeExists(_nodeId) {
    StakingNode storage node = stakingNodeMap[_nodeId];
    require(node.nodeStatus == 1, "already jailed");

    updateNodePool();
    _accrueNodeCommission(_nodeId);

    node.nodeStatus = 0;
    emit JailNode(_nodeId);
}

// resolve jailed node
function unjailNode(
    uint64 _nodeId
) external whenServer nodeExists(_nodeId) {
    StakingNode storage node = stakingNodeMap[_nodeId];
    require(node.nodeStatus == 0, "not jailed");

    updateNodePool();
    _accrueNodeCommission(_nodeId);

    node.nodeStatus = 1;
    emit UnjailedNode(_nodeId);
}
```

`jailNode` and `unjailNode` settle commission accrued before each status change, then toggle the status without recording or excluding the jailed interval. `claim`, `pendingReward`, and `claimNodeCommission` calculate rewards without checking node status, while `_updatePool` continues advancing the global accumulator. A legitimately jailed node can therefore remain jailed indefinitely while its delegators and owner claim rewards, consuming the reward reserve and potentially causing later valid claims to fail for insufficient balance.

I Recommendation

We would like to confirm with the team whether it is intended behavior.

I Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. This is intended behavior. Node jail is used to block new stake into the node, not to slash or freeze rewards for existing delegators. Existing users who already staked to the node should continue earning rewards and remain able to claim, so they are not penalized for a node-level jail event.

DGA-36 | Pre-Claim UID Binding And Allocation Logic

Category	Severity	Location	Status
Design Issue	● Informational	contracts/DgridStakePool.sol (v2-9b730c6): 641–679	● Acknowledged

Description

`preClaimAndStake` is available only while the pool is paused. When called, it consumes the caller's entire `getTdgaiAvailable` balance and calculates the pre-claim and staking amounts according to `_index`. It then marks the caller as pre-staked, adds the full available balance to `tdgaiTransferOut`, records `preInitialized[_cid][_uid]`, and emits a `PreClaimAndStake` event for subsequent off-chain processing or handling through `handlePreStake`. No tokens are transferred to `DGAISTaking` as part of this transaction.

There are two aspects of this flow that would benefit from clarification. First, `preInitialized` is keyed only by `(_cid, _uid)` and is not associated with `msg.sender`. As a result, any account with an available tDGAi balance may register an arbitrary UID and prevent the same `(_cid, _uid)` pair from being used again. It is therefore unclear whether a UID is intended to be uniquely owned by, or verified against, a specific account.

Second, for indexes `0` and `1`, the function assigns the remaining balance to `stakeAmount` after calculating `preClaimAmount`. For index `2`, however, it assigns 30% of the balance to `preClaimAmount` while leaving `stakeAmount` as zero, even though the full balance is still added to `tdgaiTransferOut`. This leaves the treatment of the remaining 70% unclear, as it is neither represented as a staking amount nor otherwise reflected in the emitted event.

Recommendation

Please clarify how UIDs are assigned, verified, and associated with user accounts, as well as whether they are intended to be unique reservations. The expected allocation for index `2` should also be documented, including how the remaining 70% is accounted for during subsequent on-chain or off-chain processing.

Alleviation

[DGrid AI, 09/15/2026]:

Acknowledged. This is intended behavior and part of an off-chain assisted migration process.

`cid` represents the exchange identifier, such as Bitget, Gate, etc. `_uid` represents the user's UID under that exchange. The pair `(_cid, _uid)` is intended to be unique and is recorded in `preInitialized` to prevent the same exchange account from being registered multiple times.

`preClaimAndStake()` does not directly transfer tokens to `DGAISTaking`. It emits a `PreClaimAndStake` event, and the backend service monitors this event, verifies the wallet/UID relationship off-chain, and then performs the corresponding processing. The migration plans combine two actions: off-chain DGAi distribution to the specified exchange UID and on-chain staking through `DGAISTaking.handlePreStake()`.

[CertiK, 09/16/2026]:

We understand that `preClaimAndStake` is an off-chain-assisted migration flow, that `(_cid, _uid)` identifies an exchange account, and that the backend verifies the wallet/UID relationship after the event is emitted.

We have two remaining points to confirm:

- 1. UID reservation and access control.** `preClaimAndStake` can be called by any account with an available tDGAi balance, and `preInitialized` is keyed only by `(_cid, _uid)`, not by `msg.sender`. As a result, an unrelated wallet could reserve a valid exchange UID, preventing the same pair from being used again on-chain even if the backend later rejects the wallet mismatch.
- 2. Residual allocation for `_index == 2`.** For `_index == 0` and `_index == 1`, the balance remaining after `preClaimAmount` is assigned to `stakeAmount`. For `_index == 2`, however, `preClaimAmount` is set to 30% of the balance while `stakeAmount` remains `0`, even though the full balance is recorded in `tdgaiTransferOut`. Could you please clarify how the remaining 70% is handled—for example, whether it is forfeited, distributed off-chain, staked through another path, or processed under a different rule?

[DGrid AI, 09/17/2026]:

Acknowledged. This was part of a one-time off-chain assisted migration campaign, and the campaign has already ended without this issue occurring in production.

For UID handling, the backend verified the relationship between the wallet address and the exchange UID before performing any off-chain processing. Although `preInitialized` is keyed by `(_cid, _uid)`, invalid wallet/UID submissions would not be honored by the backend and would not receive any off-chain distribution or staking processing.

For `_index == 2`, this is intended business logic. 30% of the user's tDGAi migration amount is airdropped as DGAi to the corresponding exchange UID. The remaining 70% tDGAi is not staked and is treated as burned/forfeited according to the migration rules. The full amount is recorded in `tdgaiTransferOut` to prevent double claiming.

Going forward, `preClaimAndStake()` is deprecated and will not be used in future migration flows. The backend no longer processes `PreClaimAndStake` events after the campaign ends.

DGA-38 Node Owners Earn Delegation Commission On Their Own Stake

Category	Severity	Location	Status
Logical Issue	● Informational	contracts/DGAIStaking.sol (v2-9b730c6): 424, 471, 1393, 1430	● Acknowledged

Description

`DGAIStaking` does not distinguish between third-party delegated positions and positions funded by the node owner. Direct staking, pre-staking, reward restaking, and DGA reward restaking all eventually call `_applyStake`, which adds the position's weight to `nodeWeightCache` without checking whether the staker is the node owner. The same issue can also arise when positions are moved between nodes through `changeNodeByUser`.

The node commission logic then treats the entire cached weight as delegated stake. `_accrueNodeCommission` calculates commission from this full amount, while `_accruePositionUnpaid` separately preserves the staker's full ordinary reward. Consequently, when a node owner stakes DGA under their own node, the owner receives both the ordinary staking reward and the commission that should apply only to third-party delegation.

The same behavior continues after an owner initiates an unstake. Although the position's active weight is removed from `nodeWeightCache`, `_pendingUnstakeUnpaid` calculates both the full cooling-period staking reward and the commission using the rate snapshot at unstake time. `_accruePendingUnstakeReward` subsequently credits both amounts when the request is settled. At high commission rates, the commission on the owner's self-stake can yield substantial rewards. The additional commission creates reward liabilities beyond the intended third-party delegation rewards and can consume the pool's reward surplus. If the surplus becomes insufficient, later reward or commission claims can revert, although the existing balance checks prevent the contract from paying rewards out of tracked principal.

Recommendation

We would like to confirm with the client whether this scenario is allowed and consistent with the intended design.

Alleviation

[DGrid AI, 09/16/2026]:

Issue acknowledged. I won't make any changes for the current version.

OPTIMIZATIONS

DGRID AI 2

ID	Title		Category	Severity	Status
DGA-32	Missing	IDGAISTaking Interface Prevents Compilation	Code Optimization	Optimization	● Acknowledged

DGA-32 | Missing IDGAISTaking Interface Prevents Compilation

Category	Severity	Location	Status
Code Optimization	● Optimization	contracts/DgridStakePool.sol (v2-9b730c6): 15	● Acknowledged

Description

`DgridStakePool.restakeReward()` imports and calls `IDGAISTaking(dgaiStaking).restakeRewardCall(...)`, but the scoped delivery does not include `Interfaces/IDGAISTaking.sol`.

As a result, the stake-pool contract cannot compile against the provided sources without an externally supplied interface stub.

Recommendation

Consider adding the `IDGAISTaking` interface file to ensure project completeness.

Alleviation

[DGrid AI, 09/16/2026]:

Issue acknowledged. I won't make any changes for the current version.

APPENDIX | DGRID AI 2

Audit Scope

dgridai/DGRID

-  contracts/libraries/TickMath8.sol
-  contracts/ChainlinkPriceFeed.sol
-  contracts/DGAIStaking.sol
-  contracts/Dgrid.sol
-  contracts/DgridAIArena.sol
-  contracts/DgridNode.sol
-  contracts/DgridPriceFeed.sol
-  contracts/DgridStakePool.sol
-  contracts/DgridTopUp.sol
-  contracts/tDGAi.sol
-  contracts/libraries/FullMath8.sol
-  contracts/libraries/SignatureValidator.sol
-  contracts/DGAI.sol
-  contracts/Proxy.sol
-  contracts/libraries/FullMath8.sol
-  contracts/libraries/SignatureValidator.sol
-  contracts/libraries/TickMath8.sol
-  contracts/ChainlinkPriceFeed.sol
-  contracts/DGAI.sol

dgridai/DGRID

-  contracts/DGAIStaking.sol
-  contracts/Dgrid.sol
-  contracts/DgridAIArena.sol
-  contracts/DgridNode.sol
-  contracts/DgridPriceFeed.sol
-  contracts/DgridStakePool.sol
-  contracts/DgridTopUp.sol
-  contracts/Proxy.sol
-  contracts/tDGAi.sol
-  contracts/libraries/FullMath8.sol
-  contracts/libraries/SignatureValidator.sol
-  contracts/libraries/TickMath8.sol
-  contracts/ChainlinkPriceFeed.sol
-  contracts/DGAi.sol
-  contracts/DGAIStaking.sol
-  contracts/Dgrid.sol
-  contracts/DgridAIArena.sol
-  contracts/DgridNode.sol
-  contracts/DgridPriceFeed.sol
-  contracts/DgridStakePool.sol
-  contracts/DgridTopUp.sol
-  contracts/Proxy.sol

dgridai/DGRID



contracts/tDGA1.sol

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

DGrid AI 2 Security Assessment | CertiK Assessed on Sept 28th, 2026 | Copyright © CertiK

